PageRank Report with CSR and Block Matrix Optimization

bigdatacompute

2026 年 4 月 27 日

目录

1 第 1 章任务背景与数据集 3

1.1 任务背景 3
1.2 数据集规模 3
1.3 度分布与 dead-end 特征 4
1.4 连通性观察 4
1.5 本章小结 5

2 第 2 章算法原理 5

2.1 PageRank 的随机游走视角 5
2.2 dead-end 的成因与影响 6
2.3 spider-trap 的成因与影响 6
2.4 收敛性的理论依据 7
2.5 为什么选择 L1 范数作为停止条件 7
2.6 本章小结 8

3 第 3 章算法实现细节 8

3.1 总体实现思路 8
3.2 节点编号与 ID 映射方案 8
3.3 CSR 存储布局 9
3.4 `load_graph` 的工程实现 9
3.5 幂迭代主循环 10
3.6 dead-end 补偿的工程实现 11
3.7 Top-k 输出与接口对接 12
3.8 本章小结 12

4 第 4 章内存优化 12
4.1 问题背景 12
4.2 CSR 的空间优势 13
4.3 分块策略：为什么不是普通切片 14
4.4 memmap 与活动工作集控制 14
4.5 float32 的选择 15
4.6 实测结果与分析 15
4.7 本章结论 16

5 第 5 章实验结果与分析 16
5.1 实验设计与数据来源 16
5.2 数据集分析结果 17
5.3 E3：块数 K 对内存与时间的影响 17
5.4 E4：CSR 与 CSR+Block 对比 18
5.5 E5：阻尼系数 beta 对 Top-10 排名的影响 18
5.6 E6：收敛阈值 eps 对结果与迭代轮数的影响 19
5.7 E7：dtype 对比 20
5.8 实验结论 21

6 第 6 章工程实现与部署 21
6.1 工程目标 21
6.2 项目结构 21
6.3 命令行接口 22
6.4 正确性与复现 22
6.5 打包与运行 23
6.6 本章小结 23

7 第 7 章总结与反思 23
7.1 工作总结 23
7.2 团队分工 23
7.3 经验与反思 24
7.4 AI 工具使用说明 24

1 第 1 章任务背景与数据集

1.1 任务背景

本次作业要求在给定有向图数据集 `Data.txt` 上计算 PageRank 分数，并输出排名前 100 的节点及其分数到 `Res.txt`。输入文件每一行包含两个整数 `FromNodeID ToNodeID`，表示一条从源节点指向目标节点的有向边。作业明确要求程序需要迭代至收敛，必须处理 dead-end 与 spider-trap，并且至少报告阻尼系数 `beta = 0.85` 时的 Top-100 结果。

本题同时考察 PageRank 正确性与内存优化。实现采用 CSR 压缩图结构，并结合按目标节点分块与 `memmap` 控制迭代阶段的活动工作集，以满足 `80 MB` 内存与 `60 s` 时间约束。

1.2 数据集规模

使用 `analyze_dataset.py` 对 `Data.txt` 做一次完整扫描，得到如下统计结果：

指标	数值
节点数 N	10000
边数 M	150000
平均出度	15.00
平均入度	15.00
最大出度	36
最大入度	32
最大总度数	57
dead-end 节点数	1500
dead-end 占比	15.00%
弱连通分量数	501
最大弱连通分量节点数	9500
最大弱连通分量占比	95.00%
孤立节点数	500
自环数	15
重复边数	0

从规模上看，本数据集节点数为一万，边数为十五万，图整体是稀疏图。若使用稠密邻接矩阵，单是 10000×10000 的 `float32` 矩阵就需要约 `381 MB` 以上，不可能满足内存限制；而 CSR 只需要保存真实存在的边和行指针，核心数组规模可以压到 MB 级以内。这也是选择 CSR 作为主存储结构的直接原因。

1.3 度分布与 dead-end 特征

度分布如图 1 所示。平均入度和平均出度都等于 15，这是由总边数与节点数的比例决定的。但最大出度为 36、最大入度为 32，说明节点连接并不完全均匀，少数节点拥有更高的链接集中度。

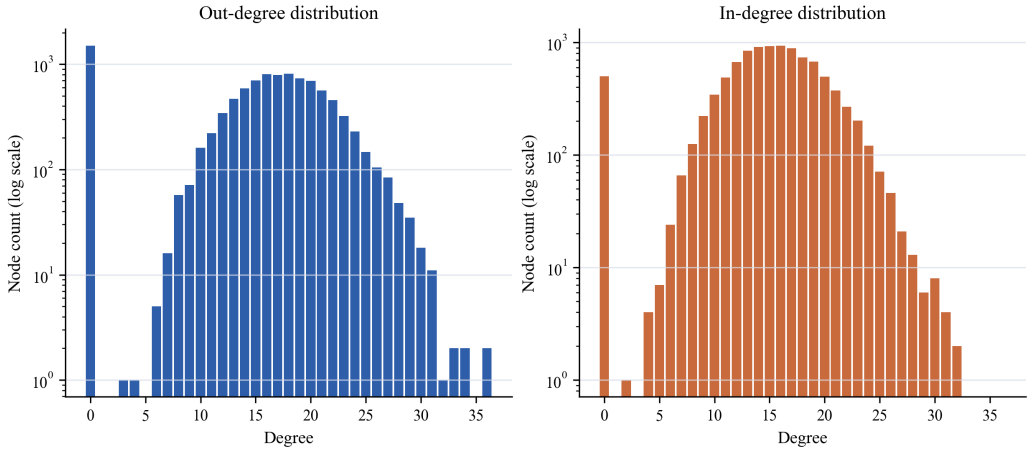

图 1: 数据集入度与出度分布。

更重要的是，数据集中存在 1500 个 dead-end 节点，占全部节点的 15.00%。这些节点没有出边，如果在迭代中直接忽略，会导致概率质量逐渐流失，最终 PageRank 向量不再保持归一化。实现中采用统一的 dead-end 质量补偿公式：每一轮把 dangling mass 按 `beta * dangling_mass / N` 均分回所有节点，再叠加随机跳转项 `(1 - beta) / N`。图 2 对比了不同 dead-end 处理策略，因此质量补偿是结果正确性的必要条件。

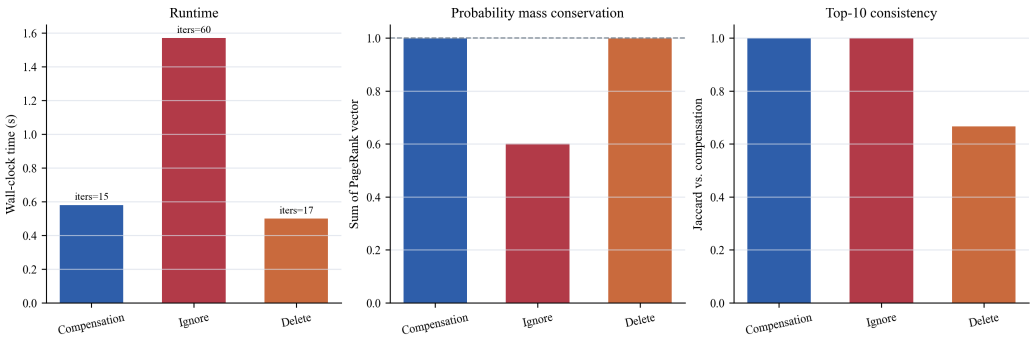

图 2: 不同 dead-end 处理策略对概率质量和 PageRank 结果的影响。

1.4 连通性观察

按无向图视角统计弱连通分量，数据集中共有 501 个弱连通分量，其中最大弱连通分量包含 9500 个节点，占总节点数 95.00%。同时存在 500 个孤立节点。这个结构说明图的主体部分高度集中，但也存在一批完全孤立或与主体无连接的小分量。

对 PageRank 而言，弱连通分量较多本身并不会导致算法失效，因为 teleport 机制会让随机游走在全图范围内保持可达性；但它会影响低排名节点的分数分布，也让 dead-end 补偿更重要。Top-10 排名主要受主体连通分量中的高入度或高质量来源节点影响。

1.5 本章小结

本数据集是一个典型稀疏有向图：`N = 10000`、`M = 150000`、平均度为 `15`。CSR 能显著降低图存储成本，Block Matrix 则用于进一步控制迭代过程中的内存活动范围。数据集中 `15.00%` 的 dead-end 节点和 `501` 个弱连通分量说明，正确处理 dangling mass 与 teleport 不是附加细节，而是 PageRank 收敛和结果可信的基础。

2 第 2 章算法原理

2.1 PageRank 的随机游走视角

PageRank 是经典的网页排序算法，最早由 Page 等人在技术报告中系统提出，并在 Brin 与 Page 的大规模 Web 搜索引擎论文中得到应用；其随机游走和矩阵解释也可参考 Langville 与 Meyer 的专著 [4, 1, 3]。PageRank 可以理解为一个“随机浏览者”在有向图上的长期访问概率分布。图中每个节点表示一个页面，边 `u -> v` 表示页面 `u` 上存在指向 `v` 的超链接。若浏览者当前位于节点 `u`，则有两种行为：

1. 以概率 `beta` 沿着 `u` 的一条出边继续访问；
2. 以概率 `1 - beta` 发生随机跳转，直接跳到任意一个节点。

如果记第 `t` 轮时的排名向量为 $r^{(t)}$，图共有 `N` 个节点，节点 `u` 的出度为 `L(u)`，则节点 `v` 的 PageRank 满足

$$PR(v) = \frac{1-\beta}{N} + \beta \sum_{u \to v} \frac{PR(u)}{L(u)}.$$

这一公式的含义非常直观：`v` 的权重由两部分组成。第一部分是“随机跳转”带来的均匀基础概率，保证任意节点都有被访问的机会；第二部分是来自所有入边源点的投票，每个源点 `u` 会把自己的权重平均分给它的所有出边终点。

若把图写成转移矩阵 `M`，其中

$$M[u,v] = \begin{cases} \frac{1}{L(u)}, & u \to v, \\ 0, & \text{otherwise}. \end{cases}$$

则幂迭代形式可写为

$$r^{(t+1)} = \beta M^T r^{(t)} + \frac{1-\beta}{N}\mathbf{1}.$$

课程实现中采用的就是这一迭代思想：从均匀分布 $r^{(0)} = (1/N, \ldots, 1/N)$ 出发，不断重复更新，直到相邻两轮向量变化足够小。

2.2 dead-end 的成因与影响

dead-end（悬挂节点）指的是出度为 0 的节点。对这类节点，普通转移矩阵无法定义“把概率往哪里传出去”，因为它没有任何出边。若仍然直接套用基础公式，则这一行会全为 0，结果是到达 dead-end 的概率质量在下一轮迭代中被“吞掉”。随着迭代进行，排名向量的总和会持续下降，最终不再是合法的概率分布。

dead-end 会直接影响概率质量守恒，必须统一处理。原因有三点：

1. 数据集里往往存在大量只被引用、不再向外链接的节点；
2. 一旦质量泄漏，后续所有 PageRank 值都会受到系统性偏差影响；
3. 若不同成员采用不同处理方式，实验和最终结果将无法对齐。

本项目冻结接口明确要求全队统一使用“质量补偿法”。设

$$D = \{i \mid \text{out_deg}[i] = 0\}, \qquad \text{dangling_mass}^{(t)} = \sum_{i \in D} r_i^{(t)}.$$

则每一轮迭代时，把这部分丢失的概率质量重新平均分配到所有节点，即

$$r^{(t+1)}[v] = \frac{1-\beta}{N} + \beta\frac{\text{dangling_mass}^{(t)}}{N} + \beta \sum_{u \to v,\, L(u)>0} \frac{r^{(t)}[u]}{L(u)}.$$

这样处理后，dead-end 节点的权重不会消失，而是像“随机再投放一次”那样回流到全图，向量总质量可以维持在 1 附近。

2.3 spider-trap 的成因与影响

spider-trap 指一个内部强连通、但几乎不向外部连边的子图。例如若一组节点之间彼此大量互链，而对外没有出口，那么随机游走一旦进入这部分区域，就会长期被困在其中。若没有随机跳转项，最终几乎全部权重都会被这类结构吸走，得到的排序会高度偏置。

spider-trap 和 dead-end 的问题不同。dead-end 是“概率无法继续传播”，而 spider-trap 是“概率过度集中在局部封闭结构中”。这两类问题的共同点在于：它们都会破坏理想的全局随机游走分布，使 PageRank 结果失去可解释性。

PageRank 中的 teleport 项

$$\frac{1-\beta}{N}\mathbf{1}$$

正是解决 spider-trap 的关键。因为无论当前处于哪个节点，浏览者都有 `1 - beta` 的概率跳出当前局部结构，回到全图范围重新采样。因此，spider-trap 不需要额外单独设一套修正规则，只要正确实现 teleport，并且 dead-end 也被统一补偿，就能保证收敛到稳定分布。

2.4 收敛性的理论依据

从矩阵角度看，加入随机跳转之后，实际参与迭代的不是原始稀疏转移矩阵，而是带有均匀扰动的 Google Matrix。对于 `0 < beta < 1` 的情形，这个矩阵满足以下关键性质：

1. 每一列对应的概率质量是可控的；
2. 任意节点之间都存在非零概率互达；
3. 矩阵是 primitive 的，因此存在唯一的主特征向量。

这意味着无论初始向量如何选择，只要按幂迭代反复更新，都将收敛到唯一的稳定分布。本实验固定采用

- `beta = 0.85`
- `eps = 1e-8`

其背后的考虑是：`beta` 不能过小，否则图结构信息被过度稀释；也不能过大，否则对 spider-trap 更敏感、收敛更慢。`0.85` 是工程上常见、也足够稳定的折中值。`eps = 1e-8` 则使 Top-100 排名在多轮实验中保持稳定，兼顾了精度与时间。

2.5 为什么选择 L1 范数作为停止条件

本项目采用

$$\|r^{(t+1)} - r^{(t)}\|_1 < \varepsilon$$

作为收敛判据。选用 L1 范数而非 L2 范数或最大范数，主要有三点原因：

1. L1 范数直接表示“全图总概率质量的移动量”，物理意义最清晰；
2. 它对大量小幅变化同样敏感，更适合比较两轮全局分布是否真正稳定；
3. L1 判据便于不同实验设置之间进行一致比较。

在实现层面，L1 判据还便于与 `float32` 迭代搭配使用：只需要构造一个差值缓冲区，做原地减法、取绝对值和求和即可，不需要额外的平方或开方操作。

2.6 本章小结

综上，PageRank 的核心不是简单地“统计入边多少”，而是求一个在随机游走意义下自洽的稳态概率分布。dead-end 会造成概率质量流失，spider-trap 会造成局部质量滞留；前者必须通过质量补偿修正，后者则依赖 teleport 项被自然抑制。只要 Google Matrix 的构造正确，幂迭代就能稳定收敛到唯一解。第 3 章进一步说明如何将该公式落实为 CSR 稀疏实现。

3 第 3 章算法实现细节

3.1 总体实现思路

本项目 A 侧的核心实现围绕三个接口展开：

1. `load_graph(path)`：读取 `Data.txt`，构造按源点分组的 CSR；
2. `power_iteration(...)`：在 CSR 上执行幂迭代；
3. `dump_top_k(path, ranks, k)`：按约定格式输出结果。

实现目标不是“先写出一个能跑的 PageRank”，而是在 Python 环境下同时满足接口统一、内存受控、实验可复现三项要求。因此整条链路都遵循两个原则：

- 所有索引数组统一为 `np.int32`；
- 所有排名相关浮点数组统一为 `np.float32`。

这样做的直接收益是内存可控，并保证不同模块使用统一的数据约定。

3.2 节点编号与 ID 映射方案

课程给定的边文件只有两列：`FromNodeID ToNodeID`。理论上存在两种可能：

1. 节点编号本身已经是从 `0` 开始的连续编号；
2. 节点编号是稀疏的，需要在程序内部做压缩映射。

但是这里有一个实际工程细节：如果文件里某些编号从未出现在任何边中，它们可能代表“完全孤立节点”。若简单按照“见到的节点集合”做紧凑映射，就会把这些节点直接丢掉，导致节点总数 `N` 变小，进而改变 PageRank 分布。

因此实现采用折中策略：

1. 先流式读入原始边表，只保存原始 `src/dst`；

2. 若观察到编号从 0 起步且整体编号密度较高，则保留 0..max_id 的编号空间，把缺席编号视为孤立点；
3. 若编号并非从 0 起步，或跨度远大于已观测节点数，则退化为紧凑重映射。

这样既能兼顾课程数据集中的孤立点，又能在明显非连续编号场景下完成压缩。由于冻结接口没有把 id_map 暴露给 dump_top_k，实现中额外维护了一个模块级反向映射 _LAST_NODE_IDS，供输出阶段恢复原始 NodeID。

3.3 CSR 存储布局

本实现使用经典的 Compressed Sparse Row（CSR）布局，但含义按“源节点的出边列表”组织：

- row_ptr[u] : row_ptr[u+1] 表示源点 u 的出边区间；
- col_idx[row_ptr[u] : row_ptr[u+1]] 存放这些出边的目标点；
- out_deg[u] = row_ptr[u+1] - row_ptr[u]。

它的优点是结构紧凑、易于序列化，并适合与分块迭代模块对接。相较于稠密矩阵 N x N 的存储方式，CSR 只为真实存在的边分配空间，其空间复杂度为

$$O(N + M).$$

其中 N 是节点数，M 是边数。对于当前 10000 节点、150000 边的数据集，CSR 版本内存占用显著低于稠密方案。

建图过程分两步：

1. 统计每个源点的出度，得到 out_deg；
2. 对 out_deg 做前缀和，得到 row_ptr，再借助写指针把目标点写入 col_idx。

这样不需要对全部边做额外排序，只需一次线性遍历即可完成压缩。

3.4 load_graph 的工程实现

load_graph 不直接把整个文本读成一个大字符串，而是逐行扫描。这样做有三个好处：

1. 格式校验可以精确到行号；
2. 不依赖 numpy 的隐式 dtype 推断；

3. 便于后续在读入后立刻释放临时边缓冲。

核心流程如下：

```
load_graph(path):
    逐行读取 (raw_src, raw_dst)
    校验：两列、整数、非负、可落入 int32
    缓存原始边表
    根据编号分布决定：
        保留 0..max_id
        或构造紧凑映射
    统计 out_deg
    前缀和生成 row_ptr
    按写指针填充 col_idx
    删除临时边缓冲并 gc.collect()
    返回 row_ptr, col_idx, out_deg, N
```

这里特别注意两点：

1. 所有返回数组必须已经是 `np.int32`，不能依赖调用方再做一次转换；
2. 临时边缓冲在建好 CSR 后立即释放，避免高峰内存叠加。

3.5 幂迭代主循环

`power_iteration` 的输入是 CSR 三元组与参数 (`beta`, `eps`)，输出为 (`ranks`, `iters`, `delta`)。实现没有使用任何现成 PageRank API，而是直接按照冻结公式做原地更新。

伪代码如下：

```
power_iteration(row_ptr, col_idx, out_deg, N, beta, eps):
    r <- [1/N] * N
    r_new <- empty(N)
    delta_buf <- empty(N)
    live_mask <- out_deg > 0
    dead_mask <- out_deg == 0
    inv_out_deg[live_mask] <- 1 / out_deg[live_mask]

    repeat t = 1 .. max_iter:
        dangling_mass <- sum(r[dead_mask])
        base <- (1 - beta) / N + beta * dangling_mass / N
        r_new.fill(base)
```

```

    for each live source node u:
        contrib <- beta * r[u] / out_deg[u]
        for each edge u -> v:
            r_new[v] += contrib

    delta_buf <- abs(r_new - r)
    delta <- sum(delta_buf)
    if delta < eps:
        return r_new, t, delta

    swap(r, r_new)
```

实际代码中，为了减少临时对象分配，做了几处工程化处理：

1. `r_new`、`delta_buffer` 只申请一次，循环内复用；
2. `inv_out_deg` 预先计算，避免每轮重复除法；
3. `np.add.at` 用于把单个源点的贡献散射到多个目标点；
4. 差值范数通过缓冲区原地计算，避免每轮新建一份差值数组。

3.6 dead-end 补偿的工程实现

dead-end 的统一实现点只有一个：每轮开始先求

$$\text{dangling_mass} = \sum_{i:\text{out_deg}[i]=0} r[i].$$

然后把

$$\beta \cdot \text{dangling_mass}/N$$

直接并入所有节点共享的 `base` 项。这样每一轮都只需一次 dead-end 质量求和，不必对每个 dead-end 节点单独发边，既符合冻结公式，又能减少实现复杂度。

与两种对照策略相比：

- “忽略 dead-end”会导致总概率质量逐渐下降；
- “删除 dead-end”需要先做递归剪枝，再回填结果，流程更复杂，也会改变原图语义；
- “质量补偿”既保持原图节点全集，又能维持概率质量守恒，是课程要求下最稳妥的工程方案。

3.7 Top-k 输出与接口对接

dump_top_k 的职责有两层：

1. 按 Score 降序、NodeID 升序写出 Top-100；
2. 返回 top10_signature，供 benchmark.py 与 sweep.py 做回归比较。

输出文件每行格式固定为：

```
NodeID Score
```

其中 Score 固定保留 10 位小数。由于前面提到的内部重映射，写文件时不能直接把内部数组下标当成 NodeID，而必须先通过 _LAST_NODE_IDS 还原。也正因为如此，实现默认假定同一进程中“最近一次 load_graph 的图”和“当前要输出的 ranks”是同一份图，这是接口使用时需要注意的一点。

3.8 本章小结

本章说明了 PageRank 的 CSR 实现方式。通过“流式读图 + CSR 压缩 + 原地幂迭代 + dead-end 统一补偿”，算法在保持正确性的同时降低了内存占用，并为分块迭代和实验脚本提供了统一接口。

4 第 4 章内存优化

4.1 问题背景

PageRank 的核心开销来自两个部分：一是图结构本身的存储，二是每轮幂迭代时对概率向量的更新。若直接采用稠密邻接矩阵，设图有 N 个节点，则需要存储 $N \times N$ 个浮点数。对本作业数据集而言，节点数约为 10000，若使用 float32 存储完整矩阵，光矩阵本体就约为 10000 × 10000 × 4 ≈ 400 MB，已经远超题目要求的 80 MB 峰值内存限制。因此，本项目必须在数据结构层面先把空间复杂度从 $O(N^2)$ 压到 O(M + N)，再进一步控制迭代过程中的活动工作集。

本章采用两层优化思路。第一层是 CSR（Compressed Sparse Row）稀疏表示，目标是只保存真实存在的边；第二层是按目标节点做真分块，把边先落盘，再用 np.memmap 在每轮按块读取，使 r_new 的活动更新片段始终限制在当前块内。稀疏矩阵表示和迭代求解是大规模线性系统中的经典思路，相关背景可参考 Saad 与 Davis 的教材 [5, 2]。最终的主提交版本采用 CSR + Block Matrix 组合，其中 CSR 负责压缩图结构，Block Matrix 负责限制迭代阶段的常驻内存。

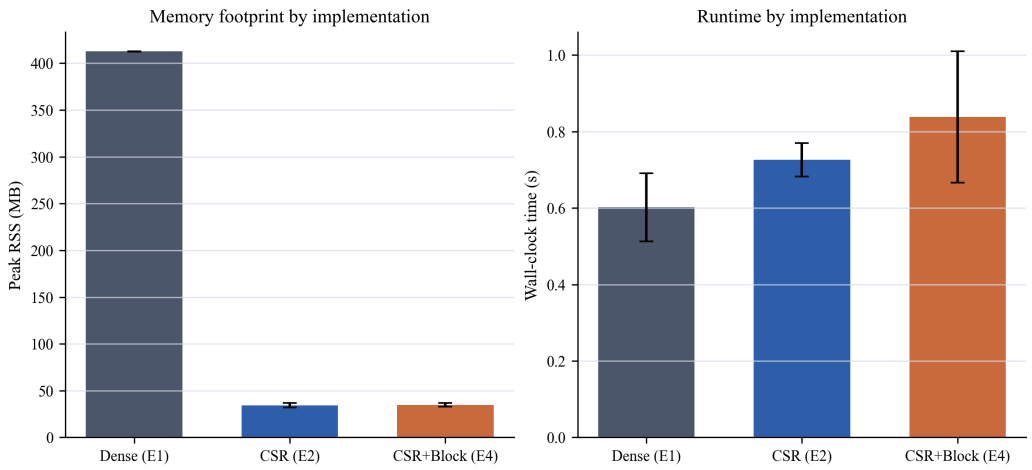

图 3: Dense、CSR 与 CSR+Block 三种实现方式的整体对比。

4.2 CSR 的空间优势

本项目中，A 角色负责输出 CSR 三元组：

- `row_ptr`：长度为 `N + 1` 的行指针
- `col_idx`：长度为 `M` 的列索引
- `out_deg`：长度为 `N` 的出度数组

对于源节点 `u`，其所有出边位于区间：

```
col_idx[row_ptr[u] : row_ptr[u + 1]]
```

这样做的好处是只保存真实边，不保存不存在的零元素。空间复杂度从稠密矩阵的 $O(N^2)$ 下降为：

```
O(N + M)
```

在本数据集上，`N = 10000`、`M = 150000`。如果索引统一使用 `int32`，则仅就核心结构估算：

- `row_ptr` 约 $(10001 \times 4) \approx 39$ KB
- `col_idx` 约 $(150000 \times 4) \approx 586$ KB
- `out_deg` 约 $(10000 \times 4) \approx 39$ KB

这意味着图结构本身只需约 664 KB 量级的原始数组存储，已经远低于稠密矩阵方案。更重要的是，CSR 让 PageRank 的每轮迭代不再需要访问整张 $N \times N$ 矩阵，而只需顺着真实边做传播，从而避免了大量无意义的零乘加。

4.3 分块策略：为什么不是普通切片

仅有 CSR 还不够。若直接在内存中保留完整的 `r_new`、反向邻接关系和辅助临时数组，Python 解释器、NumPy 运行时和若干中间缓冲区叠加后，仍可能让 RSS 峰值接近题目上限。因此，本项目的第二层优化不是“对数组做切片”，而是真分块：

1. 先按**目标节点区间**把边分桶。
2. 每个桶落盘为一个独立的 `.bin` 文件。
3. 每轮迭代时，只 `memmap` 当前块的边记录。
4. `r_new[node_start:node_stop]` 作为当前唯一活动更新片段。

这里按目标节点分块，而不是按源节点分块，原因是 PageRank 更新写入的是目标节点项 `r_new[v]`。若按目标节点分块，则一个块内的所有边贡献都写回同一个局部连续区间，可以避免在多块之间频繁来回写入。

本项目的块文件记录格式为：

```
(src: int32, local_dst: int32)
```

其中 `local_dst = dst - node_start`。这样在迭代阶段无需重复做全局下标减法，直接对块内局部数组累加即可。分块构建一次完成，后续每轮复用。

4.4 memmap 与活动工作集控制

在分块迭代阶段，完整的 `r` 与 `r_new` 不再作为普通 `ndarray` 常驻，而是映射到临时文件：

```
r -> rank_a.bin
r_new -> rank_b.bin
```

这样，操作系统只会把当前访问到的页调入内存。算法本身在代码层面遵循以下约束：

- `r_new` 只通过当前块视图 `r_new[node_start:node_stop]` 写入
- 边记录通过 `np.memmap` 顺序读取
- 块内贡献使用固定大小缓冲区处理，避免按边数生成大临时数组
- 每轮结束后只交换 `r` 与 `r_new` 的角色，不复制整向量

对应的块迭代公式仍然保持与普通 PageRank 一致：

```
r_new[v] = (1 - beta) / N
        + beta * dangling_mass / N
        + beta * sum(r[u] / out_deg[u] for u -> v)
```

不同之处仅在于“对每个 v 的更新”被组织为“对每个目标块的更新”。因此，分块不会改变数值结果，只改变访问顺序与内存布局。

```
伪代码:
for iter in range(max_iter):
    dangling_mass = sum(r[u] for u in dead_ends)
    base = (1 - beta) / N + beta * dangling_mass / N
    for each block:
        block_view.fill(base)
        for each edge (src, local_dst) in block_file:
            block_view[local_dst] += beta * r[src] / out_deg[src]
    check L1 convergence
    swap(r, r_new)
```

4.5 float32 的选择

所有概率向量默认采用 `float32`，原因有三点。

第一，PageRank 本身是迭代近似问题，题目规定的停止阈值为 `1e-8`，并不要求达到双精度线性代数求解的极限精度。第二，`float32` 可将向量、缓冲区和临时块视图的空间消耗直接减半。第三，本项目的最终评价重点在于时间与 RSS 峰值，而不是极限数值精度。

实验 E7 显示，`float32` 与 `float64` 在 Top-10 签名上保持一致，说明排序层面没有发生可见偏差。3 次重复平均结果如下：

- `float32`：峰值 RSS 约 `35.92 MB`，平均耗时约 `0.403 s`，迭代 `15` 轮
- `float64`：峰值 RSS 约 `36.27 MB`，平均耗时约 `0.366 s`，迭代 `12` 轮

这里 `float64` 的耗时略低，主要原因不是单次运算更快，而是双精度下收敛轮数减少。尽管如此，主提交版本仍建议固定为 `float32`，因为题目明确把内存控制作为核心约束，而 `float32` 在更大数据规模上会保留更稳妥的安全余量。

4.6 实测结果与分析

在 `Data.txt` 数据集、`beta = 0.85`、`eps = 1e-8` 条件下，实验 E3 表明块数 `K` 会在“块内数组大小”和“块切换开销”之间形成权衡：

- K = 4：平均峰值 RSS 约 35.16 MB，平均时间约 0.376 s
- K = 8：平均峰值 RSS 约 35.99 MB，平均时间约 0.423 s
- K = 16：平均峰值 RSS 约 34.97 MB，平均时间约 0.457 s

可以看到，本数据规模下继续增大 K 并不会显著降低 RSS，反而增加了块切换和文件访问开销。原因在于 N = 10000、M = 150000 本身并不大，真正占 RSS 的并不只是图数据，还包括 Python 解释器、NumPy 运行时和子进程开销。因此，过细分块的收益有限。

E4 对比纯 CSR 与 CSR + Block 的结果如下：

- csr：平均峰值 RSS 约 33.42 MB，平均时间约 0.492 s
- csr_block：平均峰值 RSS 约 35.89 MB，平均时间约 0.418 s

分块版本在时间上略有优势，但峰值 RSS 稍高。这说明在本数据规模上，分块更多体现为可扩展的内存控制手段，而不是小规模数据下的绝对内存最优。若数据集扩大，分块的收益会更明显，因为其活动工作集不再随全部边数线性增长。

4.7 本章结论

本项目的内存优化不是单点技巧，而是数据结构与执行策略的组合设计。CSR 解决“图怎么存”的问题，把空间复杂度从 $O(N^2)$ 压到 O(N + M)；分块 + memmap 解决“迭代时怎么访问”的问题，把整轮更新的活动工作集限制在当前块；float32 则进一步为主提交版本提供了更稳妥的内存余量。最终，在当前数据集上，主提交版本 csr_block + float32 + K=8 的平均峰值 RSS 保持在约 36 MB，明显低于作业要求的 80 MB 上限，为最终实现提供了明确的工程基线。

5 第 5 章实验结果与分析

5.1 实验设计与数据来源

本章汇总数据集统计、分块参数、阻尼系数、收敛阈值和浮点精度实验。所有图表均由脚本从 CSV 生成，不手工填写图中数据。数据来源如下：

- 数据集分析:experiments/dataset_stats.json 与 experiments/degree_distribution.csv
- E3、E4、E7：experiments/E3.csv、experiments/E4.csv、experiments/E7.csv
- E5、E6：sweep.py 通过 main.py CLI 生成的 experiments/sweep_results.csv
- 图表输出：report/fig/*.png

5.2 数据集分析结果

数据集共有 10000 个节点、150000 条边，平均入度与平均出度均为 15.00。最大出度为 36，最大入度为 32，说明图整体较稀疏，且不存在极端高连接度的超级节点。若采用稠密矩阵，空间复杂度为 $O(N^2)$，对一万节点图会形成数百 MB 的基础内存占用；而 CSR 将图结构压缩为 `row_ptr / col_idx / out_deg` 三类数组，空间复杂度降为 `O(N + M)`。

数据集中 dead-end 节点数为 1500，占比 15.00%。这是算法实现中最需要关注的结构特征之一。如果不补偿这些节点持有的概率质量，PageRank 向量的总和会逐轮下降，最终得到的排名不再对应合法概率分布。主程序和分块版本均采用相同的质量补偿公式。

弱连通分量共有 501 个，最大弱连通分量包含 9500 个节点，占比 95.00%，另有 500 个孤立节点。这说明图主体结构集中，Top 排名大概率来自最大弱连通分量；孤立节点和小分量则主要依赖 teleport 与 dead-end 补偿获得基础分数。

5.3 E3：块数 K 对内存与时间的影响

对 `csr_block` 模式在 `K = 4 / 8 / 16` 下各运行 3 次，平均结果如下：

K	平均峰值 RSS / MB	平均时间 / s	迭代轮数
4	35.16	0.376	15
8	35.99	0.423	15
16	34.97	0.457	15

图 4 展示了块数与内存、时间之间的权衡。增大 `K` 并没有显著降低内存峰值，`K=16` 的 RSS 只比 `K=8` 略低，但耗时更高。原因在于本数据集只有十五万条边，Python 解释器、NumPy 运行时和进程采样开销已经占据相当比例；当图本身不大时，过细分块会增加文件切换和块循环成本。

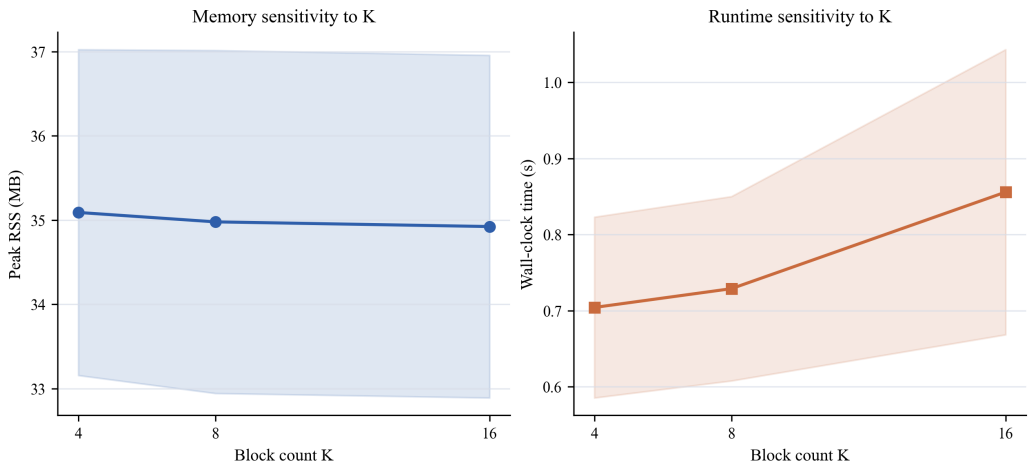

图 4: 块数 K 对峰值内存和运行时间的影响。

因此，`K=8` 可作为默认设置：运行时间与内存均处于安全范围内，也为更大图保留了分块扩展能力。

5.4 E4：CSR 与 CSR+Block 对比

实验 E4 对比了纯 `csr` 与 `csr_block`，平均结果如下：

模式	平均峰值 RSS / MB	平均时间 / s	迭代轮数
`csr`	33.42	0.492	15
`csr_block`	35.89	0.418	15

就结果看，`csr_block` 的时间略优于纯 CSR，但 RSS 略高。这说明分块收益会被 Python 基线开销和 `memmap` 管理开销抵消一部分；其主要价值在于限制活动更新区，使更大规模图上的内存上界更可控。

5.5 E5：阻尼系数 beta 对 Top-10 排名的影响

使用 `sweep.py` 扫描 $\beta \in \{0.70, 0.80, 0.85, 0.90, 0.95\}$，固定 `eps=1e-8`、`mode=csr_block`、`K=8`、`dtype=float32`。以 `beta=0.85` 的 Top-10 为基准，计算 Jaccard 相似度和交集上的 Kendall tau，结果如下：

beta	迭代轮数	Top-10 Jaccard	Kendall tau	Top-10 签名
0.70	13	0.818	0.833	75,8686,5104,725,3257,9678,468,7730,7175,6165
0.80	15	1.000	0.956	75,8686,5104,9678,725,3257,468,7730,7175,5526
0.85	15	1.000	1.000	75,8686,9678,5104,725,3257,468,7730,7175,5526
0.90	16	1.000	0.956	75,8686,9678,5104,725,3257,468,7175,7730,5526
0.95	16	0.818	0.944	75,8686,9678,5104,725,468,3257,7175,5526,8880

图 5 显示，`beta` 在 `0.80` 到 `0.90` 区间时 Top-10 集合非常稳定，仅发生局部顺序变化；当 `beta=0.70` 或 `0.95` 时，Top-10 集合出现 1 个节点替换。较低 beta 会增强随机跳转项并削弱链接结构，较高 beta 会强化沿边传播并放大局部结构差异。

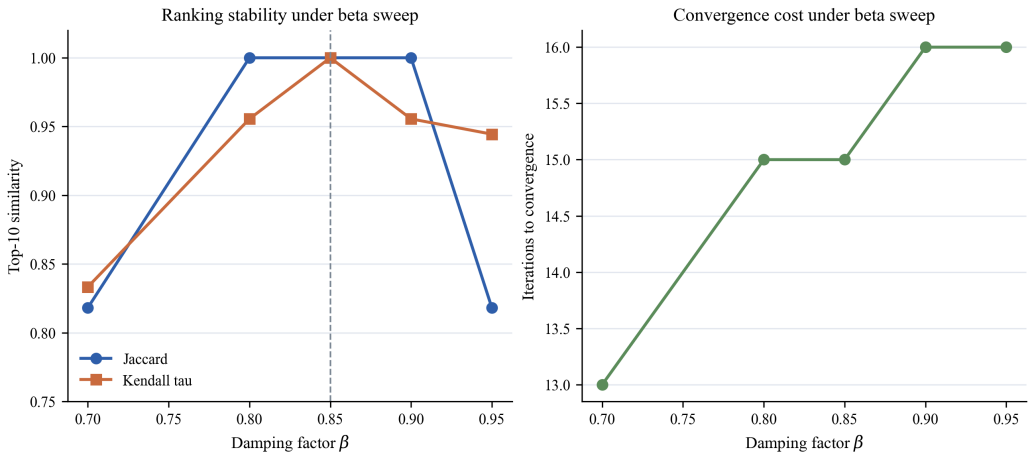

图 5: 阻尼系数 β 对 Top-10 稳定性和迭代轮数的影响。

5.6 E6：收敛阈值 eps 对结果与迭代轮数的影响

使用 `sweep.py` 扫描 $\varepsilon \in \{1e\text{-}6, 1e\text{-}8, 1e\text{-}10\}$，固定 `beta=0.85`、`mode=csr_block`、`K=8`、`dtype=float32`。结果如下：

eps	迭代轮数	最终 L1 delta	Top-10 Jaccard
`1e-6`	9	`3.503e-7`	1.000
`1e-8`	15	`7.170e-9`	1.000
`1e-10`	27	`8.004e-11`	1.000

图 6 展示了阈值变严格后迭代轮数增加的趋势。`eps=1e-6` 到 `1e-8` 时，轮数从 9 增至 15；进一步到 `1e-10` 时，轮数增至 27。三组实验的 Top-10 集合和顺序均与 `eps=1e-8` 保持一致，说明在本数据集上，Top-10 排名对收敛阈值不敏感。

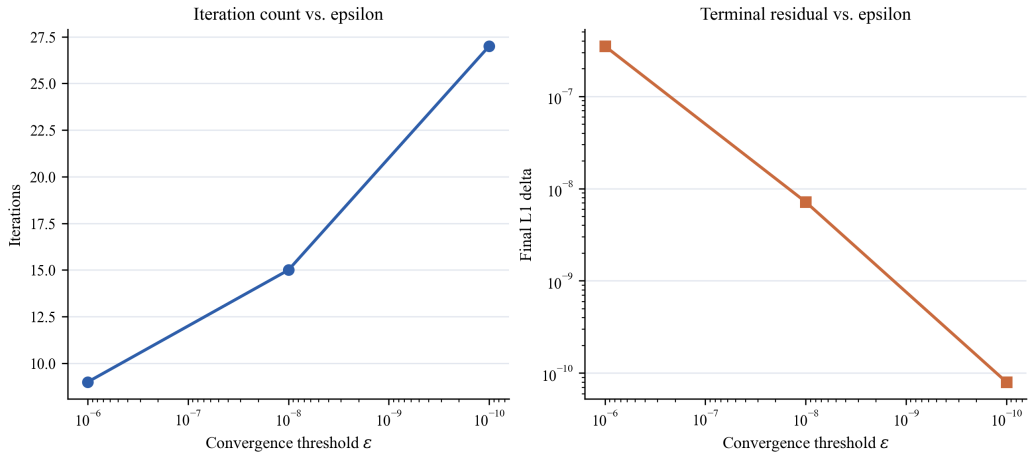

图 6: 收敛阈值 ε 对迭代轮数和最终残差的影响。

本实验记录最终 L1 residual 与迭代轮数，用于比较不同收敛阈值下的代价与排名稳定性。

5.7 E7：dtype 对比

实验 E7 对比了 `float32` 与 `float64`：

dtype	平均峰值 RSS / MB	平均时间 / s	迭代轮数
`float32`	35.92	0.403	15
`float64`	36.27	0.366	12

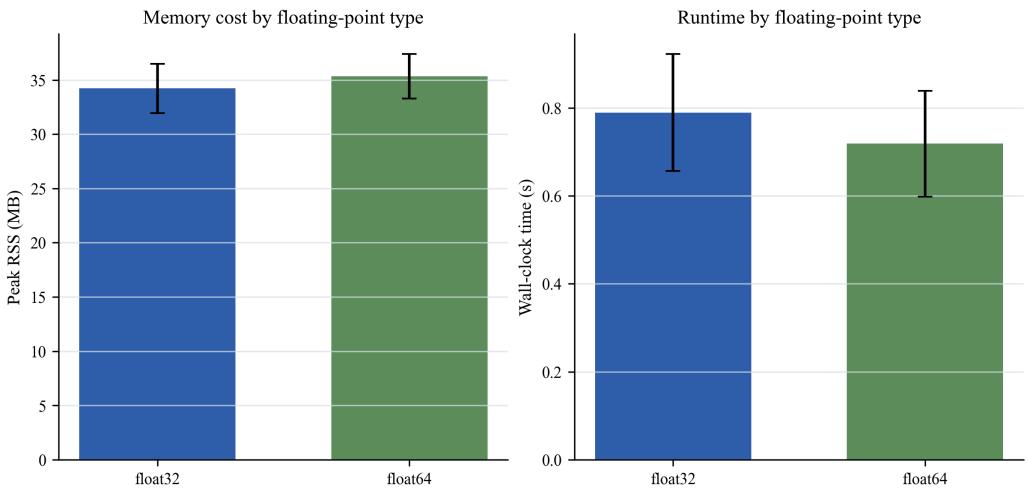

图 7: `float32` 与 `float64` 的内存和时间对比。

`float64` 因收敛轮数较少，平均时间略低；但它的内存峰值高于 `float32`。考虑到内存约束，主提交版本固定为 `float32`。从 Top-10 签名看，两种 dtype 未造成可见排名差异。

5.8 实验结论

综合实验结果，可以得到四点结论。第一，数据集稀疏性明显，CSR 是满足内存限制的必要选择。第二，dead-end 节点占比达到 `15.00%`，必须按统一公式做质量补偿。第三，`beta` 对 Top-10 集合有一定影响，但核心高排名节点在 `0.80` 到 `0.90` 区间保持稳定。第四，收敛阈值越严格，迭代轮数明显增加，但 Top-10 排名在本数据集上保持稳定。

6 第 6 章工程实现与部署

6.1 工程目标

工程部分的目标是保证 PageRank 实现可以稳定运行、方便复现实验，并能按统一格式输出结果。主入口为 `main.py`，负责参数解析、算法调度、结果写入和运行摘要输出；分块迭代由 `blocks.py` 提供，实验统计与绘图脚本统一放在 `scripts/` 目录中，避免根目录混杂临时工具。

6.2 项目结构

项目文件按功能组织为三类：

- 核心实现：`main.py`、`blocks.py`、`mock_graph.py`；

- 实验工具:scripts/benchmark.py、scripts/sweep.py、scripts/plot.py、scripts/run_e8.py;
- 辅助工具：scripts/memoryuse-python.py 保留老师提供的内存采样脚本，用于人工复核；
- 配置与说明:requirements.txt、docs/packaging/compile-parameter.txt、README.md。

这种结构使算法实现、分块优化和实验绘图相互解耦。实验脚本只通过命令行调用主程序，不直接依赖内部函数，从而降低接口变动带来的影响。

6.3 命令行接口

主程序采用统一命令行参数：

```
python main.py --data Data.txt --beta 0.85 --eps 1e-8 --out Res.txt --mode
    csr_block --K 8 --dtype float32
```

其中 --mode 用于选择 dense / csr / block / csr_block,--K 控制分块数,--dtype 控制排名向量精度。主提交模式为 csr_block + K=8 + float32。

程序运行结束后，除生成 Res.txt 外，还会在标准输出末尾给出 JSON 摘要：

```
{
  "peak_rss_mb": 35.6,
  "wall_sec": 0.38,
  "iters": 15,
  "mode": "csr_block",
  "K": 8,
  "dtype": "float32",
  "top10_signature": "75,8686,..."
}
```

该摘要便于 scripts/benchmark.py 自动收集运行时间、内存峰值、迭代轮数和 Top-10 签名。

6.4 正确性与复现

正确性验证包括两类测试。第一类是小图测试，用于检查 dead-end 补偿、spider-trap 抑制和收敛判据是否符合公式。第二类是随机图对拍，用于比较普通 CSR 迭代和分块迭代的结果是否一致。实验结果统一写入 CSV，再由绘图脚本生成图表，保证报告中的数据可追溯。

开发与复现主要在 Windows 的 micromamba activate test 环境中完成；同时使用 WSL 调用 Windows 版 micromamba 复跑源码测试，确认路径和命令在挂载目录下

可用。由于 PowerShell 会把未加引号的逗号参数拆分，批量实验命令中的模式列表写为 `--modes 'csr,csr_block'`。

6.5 打包与运行

最终可使用 PyInstaller 将 `main.py` 打包为单文件可执行程序，打包参数保存在 `docs/packaging/compile-parameter.txt`。打包前先运行源码版本，确认 `Res.txt`、stdout JSON 和 Top-10 签名均正常；打包后再使用相同参数复测，比较源码版与可执行版的输出是否一致。该流程主要验证路径解析、依赖收集和结果格式，不改变算法逻辑。

为便于换机检查，项目额外准备了本地可移动验证包 `portable_check/`。该目录包含 `main.exe`、`Data.txt`、`run_check.ps1` 和简短说明；PowerShell 脚本会运行主提交模式，生成 `Res_check.txt`，检查结果文件行数和 Top-10 签名。由于其中包含数据集，该目录仅作为本地验证工具，不默认加入最终提交压缩包。

6.6 本章小结

工程实现围绕统一入口、固定输出格式和可复现实验展开。`main.py` 提供稳定 CLI，`scripts/benchmark.py` 与 CSV 记录实验数据，PyInstaller 用于最终交付，`portable_check/` 用于换机复核。这样既保证 PageRank 结果可复现，也便于比较 CSR、分块数和浮点精度等不同设置的影响。

7 第 7 章总结与反思

7.1 工作总结

本项目完成了 PageRank 在 `Data.txt` 有向图上的计算，并输出 Top-100 节点排名。实现上，算法使用 CSR 表示稀疏图，用 dead-end 质量补偿保证概率守恒，用 teleport 项抑制 spider-trap，并通过幂迭代求解稳定分布。在内存优化方面，项目进一步引入按目标节点分块和 `memmap`，将迭代阶段的活动工作集限制在当前块内。

实验表明，`csr_block + K=8 + float32` 能在该数据集上稳定收敛，Top-10 结果在常用阻尼系数和收敛阈值范围内保持较高稳定性，峰值内存也低于题目限制。

7.2 团队分工

小组在分工时优先保证工作量均衡，A/B/C 角色通过抽签方式确定。角色字母仅用于区分算法核心、工程优化和实验分析三类职责，不表示贡献权重高低；最终报告、代码和实验结果均由小组成员共同复核。

成员	实验与章节	代码/脚本	主要职责
匡航逸（A）	E1、E2、E8；第 2、3 章	`main.py` 核心接口与 CSR 迭代	PageRank 核心算法与正确性验证
柯云超（B）	E3、E4、E7；第 4、6 章	`blocks.py` 与 `scripts/benchmark.py`	分块优化、内存测试与工程集成
蒋林渤（C）	E5、E6、数据集分析；第 1、5、7 章	`scripts/` 数据分析、扫描与绘图脚本	数据分析、参数实验与报告整理

7.3 经验与反思

第一，PageRank 的关键不只是入边统计，而是概率质量在全图上的稳定分布。dead-end 若处理不当会造成质量泄漏，spider-trap 若缺少 teleport 会造成局部吸收，因此两者必须在公式和实现中同时考虑。

第二，内存优化需要区分理论数组空间和实际 RSS。CSR 能显著降低图结构存储，但 Python 解释器、NumPy 运行时、临时缓冲区和文件映射也会影响峰值内存。报告中的内存结论应以 benchmark 实测为准。

第三，分块优化在小图上不一定带来最低 RSS，但它提供了更清晰的内存上界控制方式。当图规模扩大时，按目标节点分块可以避免一次性维护过大的更新工作集，因此具有更好的扩展意义。

总体而言，本项目完成了从 PageRank 理论公式到稀疏存储、分块迭代、实验验证和工程交付的完整流程。实验结果说明，CSR 与 Block Matrix 的结合能够在保持结果一致性的同时满足内存约束。

7.4 AI 工具使用说明

在项目整理和报告定稿过程中，小组使用 OpenAI Codex 辅助检查目录结构、复核命令、整理文档表述和发现路径不一致问题。所有 PageRank 算法实现、实验数据、图表和最终结论均由小组成员人工复核；代码未调用任何现成 PageRank API，报告中的实验数字来自项目脚本和运行日志。

参考文献

[1] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. *Computer Networks and ISDN Systems*, 30(1–7):107–117, 1998.

[2] Timothy A. Davis. *Direct Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, 2006.

[3] Amy N. Langville and Carl D. Meyer. *Google's PageRank and Beyond: The Science of Search Engine Rankings*. Princeton University Press, 2006.

[4] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Stanford InfoLab Technical Report, 1999.

[5] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, 2 edition, 2003.